

Product Requirements Document: RescueRadius

Status: Hackathon MVP

Version: 1.0

Date: 18 September 2026

Target track: Ship It preferred; Build It supported

Target geography: One Bengaluru neighbourhood, expandable to a 10 km radius

1. Product summary

RescueRadius is an expiry-aware surplus-food dispatch network for Bengaluru. It helps restaurants post safe, same-day surplus food and matches each listing with a verified NGO or volunteer that has the capacity and estimated travel time to collect it before the pickup window closes.

The map is the interface. The core product is reliable coordination: freshness window, capacity, travel time, single-winner claiming, failed-pickup reassignment, and proof of collection.

2. Problem statement

Restaurants, caterers, and institutional kitchens regularly have safe surplus food at the end of a service window. NGOs and volunteers may be able to distribute it, but coordination is often manual and time-sensitive:

- restaurants do not know who can collect immediately;
- NGOs may not have enough capacity or volunteers;
- food can become unsafe or unusable while people coordinate;
- multiple responders can claim the same donation;
- a missed pickup can cause the entire donation to be discarded.

The existence of the problem is supported by established surplus-food operations including Robin Hood Army, Swiggy Serves, and a 2026 Elio India/Robin Hood Army Bengaluru pilot. The product must therefore differentiate on last-mile dispatch reliability, not merely on listing food on a map.

Food must be described as **safe surplus food**, not plate waste. The platform records donor declarations and pickup windows but does not certify food safety or replace FSSAI-compliant

handling.

3. Product thesis

If a restaurant can post surplus food in under 30 seconds, and a verified responder can claim it only when they have sufficient capacity and can arrive before the safe pickup deadline, then more safe meals will be successfully collected instead of expiring unclaimed.

4. Goals

Hackathon goals

1. Demonstrate a complete restaurant-to-responder rescue flow.
2. Show listings updating in near real time across two browser sessions.
3. Prevent double claims and stale claims.
4. Match based on distance, capacity, food type, and remaining time.
5. Demonstrate meaningful AWS usage in the working path.
6. Produce a clear three-minute demo with seeded Bengaluru data.

User goals

Restaurant: Post surplus quickly, get a reliable pickup, and receive proof of impact.

NGO coordinator/volunteer: See only actionable donations nearby, claim one, and know the deadline and route.

Beneficiary: Receive safe food more consistently. Beneficiary identities and exact locations are not exposed in the MVP.

5. Non-goals for the 24-hour MVP

- Citywide onboarding or real NGO verification operations
- Payments or marketplace pricing
- Native Android/iOS apps
- Beneficiary accounts

- Medical or nutritional advice
- AI-based food-safety certification
- Full turn-by-turn navigation
- Automated food transport or cold-chain monitoring
- Multi-stop route optimization
- Public messaging or chat
- Integrations with Swiggy, Zomato, or restaurant POS systems

6. Personas

Restaurant operator

Works at a restaurant, bakery, caterer, or institutional kitchen. Has surplus food near the end of a service window and needs a fast way to request collection.

NGO coordinator

Manages distribution capacity and volunteers. Needs to claim donations that can actually be collected within the remaining safe window.

Volunteer responder

Can collect food locally. Needs a precise pickup location, quantity, deadline, and confirmation flow.

Platform administrator

Seeds verified demo organizations, reviews suspicious listings, and views operational metrics.

7. MVP user journey

Restaurant journey

1. Sign in as a restaurant.
2. Select a saved restaurant location.
3. Enter food type, quantity, dietary category, packed time, and pickup deadline.

4. Confirm the safe-surplus checklist.
5. Publish the listing.
6. See the listing status change from AVAILABLE to CLAIMED , PICKED_UP , and DELIVERED .
7. Receive a completion record with meals rescued and timestamps.

NGO/volunteer journey

1. Sign in as an NGO or volunteer.
2. Open the live map centered on the demo neighbourhood.
3. View available listings within 10 km.
4. Filter by food type, quantity, dietary category, and time remaining.
5. Open a listing and see estimated travel time and pickup deadline.
6. Claim the listing.
7. Confirm pickup with an OTP or QR code.
8. Confirm delivery or mark the pickup as failed.

Failure journey

1. A responder claims a listing.
2. The responder cancels or misses the pickup deadline.
3. The system returns the listing to AVAILABLE if time remains.
4. The system notifies or exposes it to the next eligible responder.
5. If the deadline passes, the listing becomes EXPIRED and cannot be claimed.

8. Functional requirements

P0 — must work

FR-001: Role-based access

The system must support three roles:

- RESTAURANT

- RESPONDER
- ADMIN

The MVP may use seeded accounts. Cognito is preferred for Ship It; a local mock-auth adapter is acceptable for Build It.

FR-002: Create surplus listing

A restaurant must be able to create a listing with:

- restaurant name and location;
- food description;
- quantity in meals or portions;
- vegetarian/non-vegetarian/contains-allergen category;
- packed timestamp;
- pickup deadline;
- optional photo;
- donor safety acknowledgement.

FR-003: Live availability map

Responders must see available listings plotted on a Bengaluru map. Listings must be filtered to a 10 km radius from the responder's selected location.

The map must distinguish at least:

- available;
- claimed;
- expiring soon;
- expired.

FR-004: Eligibility-aware matching

The system must rank listings or responders using:

1. remaining time before pickup deadline;
2. estimated travel time or distance;
3. responder capacity;
4. dietary/food-type compatibility;
5. verified responder status.

The 10 km radius is only a coarse boundary. A responder must not be recommended solely because they are geographically close if the estimated travel time exceeds the remaining safe window.

FR-005: Atomic claiming

Only one responder can claim a listing. A second claim attempt must fail gracefully and show that the listing is no longer available.

FR-006: Status lifecycle

The listing lifecycle must be:

AVAILABLE → CLAIMED → PICKED_UP → DELIVERED
↘ CANCELLED / EXPIRED

Every transition must store the actor and timestamp.

FR-007: Expiry enforcement

Expired listings must not appear as claimable. The backend must enforce expiry even if the frontend has stale data.

FR-008: Pickup confirmation

The restaurant and responder must be able to complete a simple OTP or QR-based pickup confirmation.

FR-009: Notifications

The system must notify the relevant responder and restaurant when:

- a listing is claimed;
- a claim is cancelled;
- a listing is close to expiry;
- pickup is confirmed;
- a pickup fails.

For the demo, an in-app notification plus SNS email or SMS is sufficient.

FR-010: Impact dashboard

The admin view must show:

- total meals listed;
- meals claimed;
- meals picked up;
- meals delivered;
- expired listings;
- average time to claim;
- pickup success rate.

P1 — useful if time remains

- Search by neighbourhood.
- Restaurant donation history.
- Responder capacity profile.
- Photo upload to S3.
- WhatsApp share link.
- Basic admin approval of restaurants and NGOs.

P2 — explicitly defer

- Route optimization across multiple pickups.
- Machine-learning demand prediction.

- Automated identity or NGO-document verification.
- Temperature sensors.
- Payments, tax receipts, or CSR accounting.

9. Food-safety and trust requirements

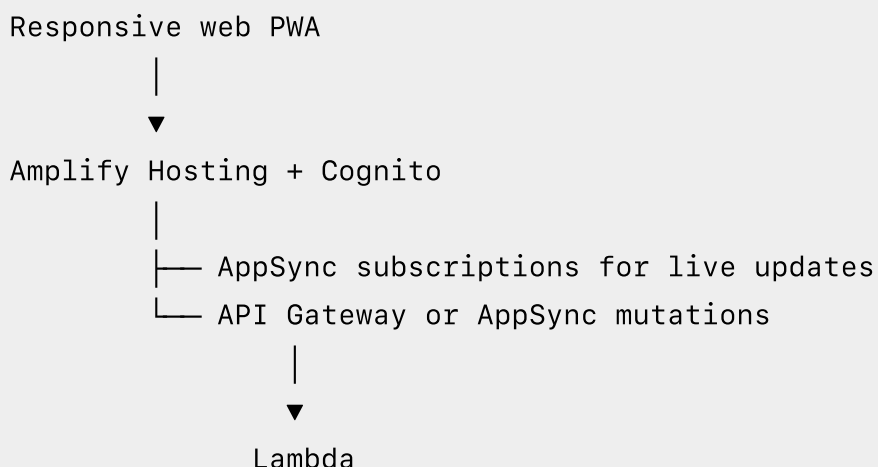
The MVP must use the following language and safeguards:

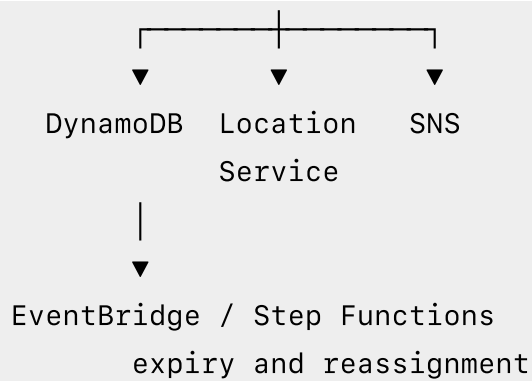
- “Safe surplus food” means food that is unserved and declared suitable for donation by the donor.
- The platform does not guarantee food safety.
- Donors must provide packed time and pickup deadline.
- The system must prevent claims after expiry.
- Exact beneficiary locations must not be shown publicly.
- Public map markers should expose only the restaurant pickup location and non-sensitive listing data.
- Demo data must be synthetic unless a real partner explicitly agrees to participate.

FSSAI guidance includes requirements around recovery agencies, hygienic handling, transport time, and hot/cold temperature control. The writeup should acknowledge these constraints rather than claiming that the prototype solves the full compliance problem.

10. Suggested AWS architecture

Ship It architecture





AWS responsibilities

- **Amplify Hosting:** host the responsive web app.
- **Cognito:** authenticate seeded restaurant and responder roles.
- **API Gateway or AppSync:** expose the application API.
- **Lambda:** create listings, match, claim, transition status, and calculate metrics.
- **DynamoDB:** store listings, users, claims, status history, and capacity.
- **Amazon Location Service:** map display, geocoding, and optional travel-time estimates.
- **AppSync subscriptions:** push listing and claim updates to connected clients.
- **EventBridge or Step Functions:** expire listings and trigger reassignment.
- **SNS:** send alerts for claims, expiry, and failed pickups.
- **CloudWatch:** logs and basic operational metrics.
- **S3:** optional listing photos and exported impact reports.

Build It architecture

Use a shared frontend and domain layer with local adapters:

- SAM CLI or LocalStack for AWS-compatible local functions;
- local DynamoDB or SQLite adapter;
- deterministic Haversine distance calculation;
- seeded map data;
- mocked notification adapter.

The project must still visibly use an AWS open-source project or AWS service in accordance with the hackathon rules.

11. Data model

User

```
{
  "id": "user-123",
  "role": "RESPONDER",
  "name": "Hope Kitchen NGO",
  "verified": true,
  "location": { "lat": 12.9352, "lng": 77.6245 },
  "capacityMeals": 40,
  "foodPreferences": ["VEG", "PACKAGED"],
  "active": true
}
```

Surplus listing

```
{
  "id": "surplus-456",
  "restaurantId": "restaurant-001",
  "foodDescription": "Packed rice and dal meals",
  "quantityMeals": 30,
  "foodCategory": "VEG",
  "packedAt": "2026-09-18T18:00:00Z",
  "pickupDeadline": "2026-09-18T19:00:00Z",
  "location": { "lat": 12.9352, "lng": 77.6245 },
  "status": "AVAILABLE",
  "claimedBy": null,
  "createdAt": "2026-09-18T18:05:00Z"
}
```

Status event

```
{
  "listingId": "surplus-456",
  "from": "AVAILABLE",
  "to": "CLAIMED",
  "actorId": "user-123",
}
```

```
"timestamp": "2026-09-18T18:12:00Z",  
"reason": null  
}
```

12. API surface

```
POST    /listings  
GET     /listings?lat={lat}&lng={lng}&radiusKm=10  
GET     /listings/{id}  
POST    /listings/{id}/claim  
POST    /listings/{id}/cancel  
POST    /listings/{id}/pickup  
POST    /listings/{id}/deliver  
GET     /me  
GET     /dashboard/impact
```

The claim endpoint must use a conditional write or transaction so that two responders cannot successfully claim the same listing.

13. Non-functional requirements

- Mobile-first responsive UI.
- Core listing and claim actions complete in under three seconds in the demo environment.
- Listing updates visible to another connected browser within five seconds.
- Backend expiry enforcement must work independently of frontend state.
- No secrets committed to the repository.
- No real beneficiary personal data in demo fixtures.
- Clear error states for expired, already claimed, and unavailable listings.
- All seeded data clearly labelled as demo data.

14. Success criteria for the hackathon demo

The project is successful if a judge can see, without explanation:

1. A restaurant creates a listing in under 30 seconds.

2. The listing appears on a second client's map.
3. A responder sees the distance, capacity fit, and remaining time.
4. The first responder claims it successfully.
5. A second responder is prevented from claiming it.
6. A cancellation or missed pickup causes reassignment.
7. An expired listing cannot be claimed.
8. The dashboard records the rescued meal.
9. At least one AWS service is visibly part of each demonstrated workflow.

15. Three-minute demo script

0:00–0:25 — Problem

Show a restaurant with 30 safe surplus meals and 45 minutes before the pickup deadline. Explain that the challenge is not discovering food waste; it is getting the right responder there in time.

0:25–1:10 — Restaurant flow

Create and publish the listing. Show the packed time, deadline, quantity, location, and safety acknowledgement.

1:10–1:45 — Live matching

Switch to the responder browser. Show the listing appearing on the map, with distance, capacity match, and time remaining.

1:45–2:15 — Reliable claiming

Claim it from responder A. Attempt to claim it from responder B and show the atomic rejection.

2:15–2:40 — Failure recovery

Cancel responder A or simulate a missed pickup. Show the listing becoming available again and being claimed by responder B.

2:40–3:00 — Impact and AWS

Show the delivered status, impact metric, event history, and a simple architecture overlay naming DynamoDB, Lambda, AppSync, Location Service, EventBridge, and SNS.

16. 24-hour implementation plan

Hours 0–2: foundation

- Confirm one neighbourhood and seeded coordinates.
- Create repository and shared domain types.
- Build fixture users, restaurants, NGOs, and listings.
- Decide Build It or Ship It deployment target.

Hours 2–6: core backend

- Create listing endpoint.
- Create listing query with radius filter.
- Implement status lifecycle.
- Implement conditional claim.
- Add expiry checks.

Hours 6–10: frontend flow

- Restaurant create-listing form.
- Responder map and listing cards.
- Claim and status controls.
- Mobile responsive layout.

Hours 10–14: realtime and notifications

- Add AppSync subscription or short-interval polling.
- Add SNS notification or in-app notification adapter.
- Add cancellation and reassignment.

Hours 14–17: AWS deployment

- Deploy frontend.
- Deploy Lambda/API and DynamoDB.
- Configure Cognito or seeded demo auth.
- Configure CloudWatch logs.

Hours 17–20: verification

- Test double claim.
- Test expired listing.
- Test failed pickup.
- Test two-browser realtime update.
- Test mobile layout.

Hours 20–24: submission

- Record the three-minute demo.
- Write the problem, impact, architecture, and limitations.
- Add setup instructions and architecture diagram.
- Confirm the public repository and video work while signed out.

17. Risks and mitigations

Risk	Mitigation
Existing competitors make the idea look copied	Position it as expiry-aware dispatch and failed-pickup recovery, not a food map
Food-safety liability	Use safe-surplus language, donor acknowledgement, expiry enforcement, and no claims of certification
No real partner data	Use synthetic data and explicitly label it; validate assumptions in the writeup
AWS deployment takes too long	Build the vertical slice locally first; deploy only the working path
Map integration becomes a time sink	Use preset coordinates and a simple map; add Amazon Location routes only after the core flow works

Risk	Mitigation
Real-time infrastructure is too complex	Use polling as a fallback; real-time means state becomes visible quickly, not necessarily WebSockets
NGO cold-start problem	Seed verified responders and explain that an NGO partnership is the next validation step
Public exposure of vulnerable communities	Never expose beneficiary names or exact distribution locations

18. Open questions after the hackathon

- Which Bengaluru NGO would own the operational queue?
- Do restaurants prefer donation, discounted sale, or CSR credit?
- What is the true safe pickup window for different food categories?
- Who pays for volunteer travel or transport?
- What verification documents should NGOs provide?
- Can the system integrate with restaurant POS or kitchen surplus reports?
- Is the bottleneck supply, volunteers, transport, or recipient capacity?

19. Research and references

- [Bharat Builds / First Commit criteria](#)
- [Robin Hood Army](#)
- [Swiggy Serves and Robin Hood Army](#)
- [Elior India and Robin Hood Army Bengaluru pilot, March 2026](#)
- [Perfectly Good](#)
- [ResQBag Bengaluru](#)
- [Food Rescue Map](#)
- [AharaSetu](#)
- [SaveFood](#)
- [FSSAI Save Food Share Food](#)
- [FSSAI surplus-food handling guidance](#)
- [Amazon Location Service](#)

- AWS AppSync real-time data